

用 React + Three.js 做了一个细胞 3D 教学网页，上课再也不用 PPT 截图了

原创 柳杉前端 柳杉前端 2026年5月16日 08:15

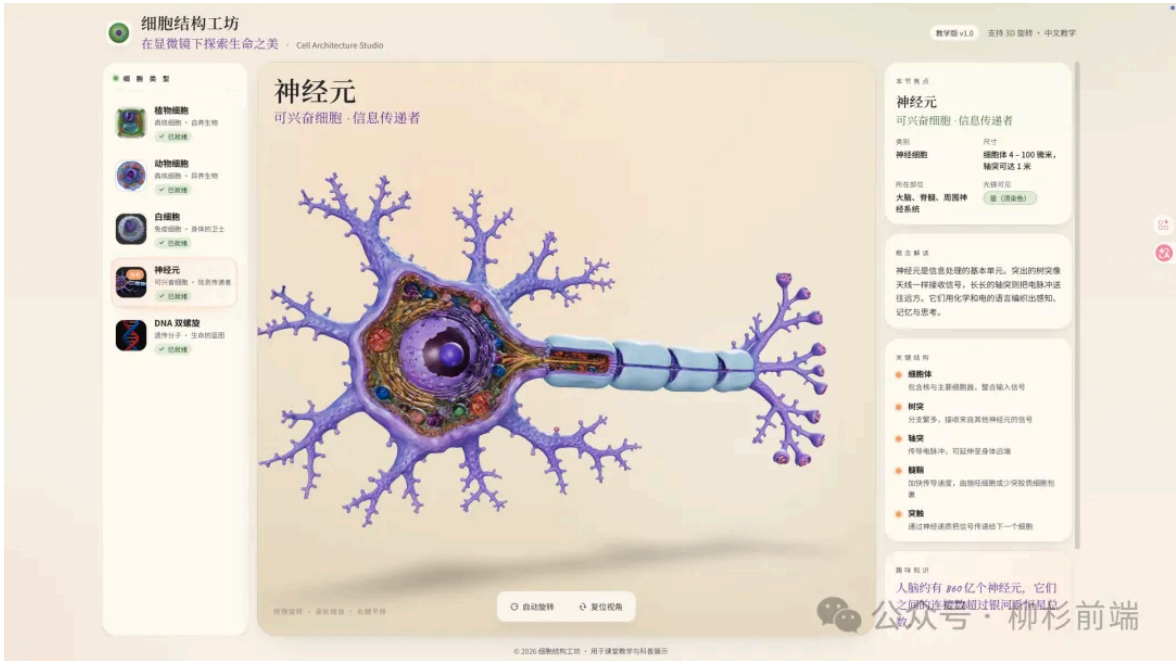
生物课上讲细胞结构，老师通常的做法是打开课件翻到一张平面示意图，然后指着“细胞膜在外面，细胞核在里面”讲上半小时。学生看得迷糊，老师讲得乏味。

有没有可能让学生自己动手转一转这个细胞，从任意角度看清楚每一层结构？这个叫 **LearningCell** 的开源项目就是干这件事的。它是一个跑在浏览器里的交互式 3D 生物教学网页，5 个精细的细胞和分子模型——植物细胞、动物细胞、白细胞、神经元、DNA 双螺旋——全部可以拖动旋转、捏合缩放、任意观察。不需要安装任何客户端，开个链接就能用。



项目有什么

整个项目的核心是 5 个经过 Draco 压缩的 **.glb** 三维模型，每个文件体积控制在 6 到 11 MB 之间，在正常网络下秒级可用。



植物细胞：包含细胞壁、叶绿体、大液泡等植物特有结构，层次清晰，可以从外壁逐层往里看。

动物细胞：包含细胞膜、内质网、高尔基体等，与植物细胞对比展示效果很直观。

白细胞：作为免疫系统的代表模型，外形和内部结构都有还原。

神经元：树突、轴突、突触的空间关系是平面图最难表达清楚的，3D 模型解决了这个问题。

DNA 双螺旋：双螺旋骨架加上碱基对，旋转一圈就能看懂什么叫"双链"。

页面整体是顶栏 + 三栏主体 + 底栏的结构。顶栏显示品牌标题和教学版本信息；主体区域为侧栏 + 主视图 + 信息面板三栏布局——左侧是模型列表，点击切换；中间是 WebGL 渲染的 3D 视口，鼠标或触摸操控；右侧显示当前模型的生物学说明；底栏标注版权和用途。

技术选型

项目的技术栈选得很现代，但每一步都有具体原因。

```
Vite + React 19 + TypeScript  
three.js / @react-three/fiber / @react-three/drei  
DRACOLoader (本地 wasm 解码器)  
fetch + ReadableStream (自定义流式加载)
```

为什么用 `@react-three/fiber` 而不是直接操作 `three.js`?

直接用 `three.js` 写场景，每一个灯光、每一个相机、每一次 `resize` 都要手动管理生命周期，放进 `React` 组件里很容易搞出内存泄漏。`@react-three/fiber` 把 `three.js` 的命令式 API 包成 `React` 的声明式组件，场景里的物体就是 `JSX`，卸载组件自动清理 `WebGL` 资源，省了大量心智负担。

Draco 解码器为什么要本地自带?

`DRACOLoader` 默认会从 `Google` 的 `CDN` (`gstatic.com`) 加载 `wasm` 解码器，在国内访问这个地址通常超时或者直接失败。项目把解码器文件放在 `app/public/draco/` 目录下，部署到任何地方都能离线运行，完全不依赖外部 `CDN`。

核心代码解析

1. 数据层： `data/models.ts`

所有模型的元数据集中在这一个文件里管理：

```
export const MODELS: CellModel[] = [  
  {  
    id: 'plant-cell',  
    name: '植物细胞',  
    modelUrl: asset('models/plant-cell.glb'),  
    imageUrl: asset('images/plant-cell.jpg'),  
  },  
]
```

```

    fileSize: 6030628,
    description: '植物细胞是构成植物体的基本单位.....',
    // subtitle, category, accent, size, features, funFact 等字段
  },
  {
    id: 'animal-cell',
    name: '动物细胞',
    modelUrl: asset('models/animal-cell.glb'),
    imageUrl: asset('images/animal-cell.jpg'),
    fileSize: 10673912,
    description: '动物细胞缺少细胞壁和叶绿体.....',
  },
  // ...其余三个模型
];

```

每个模型条目包含展示用的元信息（`name`、`subtitle`、`category`、`features`、`funFact` 等）和加载用的工程字段（`modelUrl`、`fileSize`、`defaultRotationY`、`displayScale`）。`asset()` 工具函数通过 `import.meta.env.BASE_URL` 自动拼接部署路径前缀，确保不同部署环境下资源路径正确。

侧边栏列表、3D 视口、信息面板三个组件都从这里取数据，互相解耦。想新增一个模型，只需要往数组里加一条记录，其他地方自动响应，不需要改 UI 代码。

2. 加载层：lib/modelLoader.ts

这是整个项目最值得细看的部分。作者没有直接用 `GLTFLoader` 的标准 `load()` 方法，而是自己写了一个基于 `fetch + ReadableStream` 的流式下载器。

为什么要这样做？

`GLTFLoader.load()` 内部的进度回调依赖 `XMLHttpRequest` 的 `onprogress` 事件，但这个事件只有在服务器返回了正确的 `Content-Length` 响应头时才能算出真实百分比，否则只能显示已下载字节数，无法显示“已完成 73%”这样的进度。用 `fetch + ReadableStream` 则可以逐块读取响应体，精确统计下载进度。

核心逻辑大致如下：

```

async function fetchWithProgress(
  url: string,
  expectedSize: number,
  onProgress: (loaded: number, total: number) => void
): Promise<ArrayBuffer> {
  const response = await fetch(url, { cache: 'force-cache' });
  if (!response.ok) {
    throw new Error(`下载失败: ${url} (${response.status})`);
  }
  const headerTotal = Number(response.headers.get('Content-Length')) || 0;
  const total = headerTotal > 0 ? headerTotal : expectedSize;

  if (!response.body) {

```

```
const buffer = await response.arrayBuffer();
onProgress(buffer.byteLength, total || buffer.byteLength);
return buffer;
}

const reader = response.body.getReader();
const chunks: Uint8Array[] = [];
let loaded = 0;

while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  if (value) {
    chunks.push(value);
    loaded += value.length;
    onProgress(loaded, Math.max(total, loaded));
  }
}

const buffer = new Uint8Array(loaded);
let offset = 0;
for (const chunk of chunks) {
  buffer.set(chunk, offset);
  offset += chunk.length;
}
return buffer.buffer;
}
```

下载完成后，把拿到的 `ArrayBuffer` 喂给 `GLTFLoader.parse()`（回调式 API，代码中用 Promise 包装），再经 Draco 解码，最终得到可以直接挂进 React Three Fiber 场景的 `GLTF` 对象。

值得注意的是，`fetchWithProgress` 接受一个 `expectedSize` 参数——这是写在 `models.ts` 里的文件体积。当服务器未返回 `Content-Length` 头（比如某些 CDN 或开发服务器）时，用它做兜底分母，保证进度条依然能显示真实百分比，而不是只跳 0 和 100。

缓存策略

`fetch` 请求加了 `cache: 'force-cache'`，这意味着浏览器会把 `.glb` 文件缓存下来，下次访问同一个模型几乎是瞬开。代码层面还额外维护了一个内存 `Map<string, LoadEntry>`，以模型 URL 为键缓存加载状态和解析结果，避免同一个会话里重复下载和解析同一个文件。

3. 调度层：加载优先级控制

模型文件合计约 40 MB 出头，全部并发下载会打架——尤其是首屏那个植物细胞还没加载完，带宽就被其他四个瓜分了。

项目的解决方案是一套串行队列加优先级覆盖的调度机制：

用户进入页面

└─ 立即下载植物细胞（默认首选，约 6 MB）

- └ 解析完成 OR 超时 5 秒（兜底）
 - └ 按顺序串行下载剩余 4 个模型

当用户在剩余模型还没下完时点击了侧栏里的某个模型，这个模型的下载会立刻被提升到前台，显示进度条，队列里的其他任务暂停让路。切换体验比较丝滑，不会出现"点了半天没反应"的情况。

4. 状态层： `hooks/useModel.ts`

加载状态的管理通过自定义 Hook 暴露给组件：

```
export function useModel(url: string, opts: Options): UseModelState {
  const { autoStart = true, fileSize } = opts;
  const [, forceUpdate] = useReducer((n: number) => n + 1, 0);

  useEffect(() => {
    let unsubCache: (() => void) | null = null;
    let unsubEntry: (() => void) | null = null;

    const attach = () => {
      const entry = getLoadEntry(url);
      if (!entry) return false;
      // 取消全局缓存监听，切换到 entry 级局部监听
      if (unsubCache) { unsubCache(); unsubCache = null; }
      const listener = () => forceUpdate();
      entry.listeners.add(listener);
      unsubEntry = () => entry.listeners.delete(listener);
      return true;
    };

    if (autoStart) loadModel(url, { fileSize });

    if (!attach()) {
      // entry 尚不存在时，订阅缓存增长事件等待它创建
      unsubCache = subscribeCache(() => {
        if (attach()) forceUpdate();
      });
    }

    return () => { unsubCache?.(); unsubEntry?.(); };
  }, [url, fileSize, autoStart]);

  const entry = getLoadEntry(url);
  if (!entry) return { status: 'idle', progress: 0 };
  return { status: entry.status, progress: entry.progress, entry };
}
```

这个设计把加载逻辑和 UI 渲染彻底分开。`modelLoader` 模块是全局单例，维护所有模型的下载/解析状态；`useModel` 通过监听 `LoadEntry.listeners`（entry 级）和 `subscribeCache`（全局级）两层事件来感知状态变化，用 `useReducer` 强制触发重渲染。`autoStart` 参数让调用方可以选择是否立即触发下载——`App.tsx` 中的编排逻辑正是利用 `autoStart=false` 的观察模式来控制预加载顺序。

5. 渲染层：3D 视口

三维场景用 `@react-three/fiber` 声明，`@react-three/drei` 提供了几个常用工具：

```
<Canvas shadows="percentage" dpr={[1, 2]} camera={{ position: [0, 0, 4.4], fov: 45 }}>
  <ambientLightintensity={0.55} />
  <directionalLightposition={[5,6, 4]} intensity={1.1}castShadow />
  <directionalLightposition={[-3,2, -4]} intensity={0.35} />

  <Suspensefallback={null}>
    <Environmentpreset="studio"environmentIntensity={0.55} />
  </Suspense>

  {isReady && <ModelSceneglTF={entry.gltf}autoRotate={false}
  initialRotationY={model.defaultRotationY}displayScale={model.displayScale} />}

  <ContactShadows position={[0, -1.35, 0]} opacity={0.32} scale={6} blur={2.4} />
  <OrbitControlsenableDampingdampingFactor={0.08}minDistance={1.5}maxDistance={9} />
</Canvas>

{/* 进度蒙层在 Canvas 外部渲染 */}
{!isReady && <ProgressOverlayprogress={progress}status={status}modelName={model.name} />}
```

`OrbitControls` 处理鼠标拖拽旋转和滚轮缩放，`enableDamping` 打开惯性阻尼，松开鼠标后模型还会顺着惯性转一小段再停，操控感比“松手即停”要舒服很多。`Environment preset="studio"` 是 `drei` 提供的 HDR 环境光预设，配合 `environmentIntensity` 控制反射强度。场景里配了两盏方向光（主光 + 补光）以及 `ContactShadows` 接触投影，一起让模型呈现出自然的光泽和地面阴影。

模型并不是直接用 `<primitive object={gltf.scene} />` 挂进场景，而是通过 `<ModelScene>` 组件做了归一化处理：克隆场景 → 计算包围盒 → 居中 → 按统一目标尺寸缩放（再乘以每个模型各自的 `displayScale`），这样不同体积的模型切换后在视口里的呈现大小保持一致。加载进度也不是放在 `Suspense fallback` 里，而是以绝对定位蒙层（`<ProgressOverlay>`）覆盖在 `Canvas` 上方，避免 `Canvas` 反复挂载/卸载带来的 `WebGL` 上下文重建开销。

部署：GitHub Actions 全自动

项目内置了 `.github/workflows/deploy.yml`，推代码到 `main` 分支就自动构建并发布到 GitHub Pages，整个过程不需要手动干预。

部署流程关键的一步是构建时注入 `VITE_BASE` 环境变量：

```
-name: Build
run: |
  REPO_NAME="${GITHUB_REPOSITORY#*/}"
  VITE_BASE="/${REPO_NAME}/" npm run build
```

```
touch dist/.nojekyll
```

构建脚本先从 GitHub 内置的 `GITHUB_REPOSITORY` 环境变量（格式为 `owner/repo`）里截取仓库名，然后把它作为 `VITE_BASE` 内联传给 Vite。 `vite.config.ts` 中通过 `process.env.VITE_BASE || './'` 读取这个值作为 `base`。部署在 `username.github.io/LearningCell/` 这样的子路径下时，图片和模型文件的 URL 前缀必须是 `/LearningCell/`，不然资源全部 404。改成自定义域名或用户主页根目录部署时，把这个值换成 `"/` 就好。 `touch dist/.nojekyll` 则是告诉 GitHub Pages 不要走 Jekyll 构建，保留以下划线开头的文件。

可以怎么扩展

这套架构本身并不局限于生物课。流式加载 + 优先级调度 + Draco 压缩这套组合，适合任何需要在网页里展示多个较大 3D 模型的场景：地理课的地形模型、化学课的分子结构、工业产品的 3D 说明书.....

如果想加新模型，只需要：

1. 把 `.glb` 文件用 Draco 压缩后放进 `app/public/models/`
2. 在 `data/models.ts` 里追加一条记录
3. 准备一张缩略图放进 `app/public/images/`

三步完成，其他什么都不用动。

模型可以怎么来，你可以使用腾讯混元3D通过上传图片生成glb格式模型。

网站地址：<https://3d.hunyuan.tencent.com/>

blender打开这个模型，发现它的布线太多了，很难对其修改。

小结

LearningCell 是个面向课堂场景打磨得比较细致的小工具。技术层面没什么特别新的东西，three.js 和 React 都是成熟方案，但作者在细节上做了不少工程上的取舍：本地化 Draco 解码器解决国内 CDN 问题、流式进度解决"假进度条"问题、串行调度解决首屏带宽争抢问题。这些细节放在一起，让一个看起来简单的展示页面实际用起来比预期要顺畅。

项目地址：<https://github.com/GordenSun/LearningCell>

柳杉前端
邀请你前往腾讯公益一起捐

点亮“心”希望

一起捐

可视化 · 目录

上一篇

下一篇

2026/6/25 20:49

用 React + Three.js 做了一个细胞 3D 教学网页，上课再也不用 PPT 截图了

海量开源模版免费用！所有交互开发点点鼠标
就行！这款国产数字孪生软件绝了！

一个冷门 **Three.js** 仓库，解决了工业**3D**大屏
80% 的脏活